Classifying Threat Intelligence Sentences into MITRE ATT&CK Tactics

AIC211 – AI for Cybersecurity · Project Topic 10: Threat Intelligence / ATT&CK Technique Classification

Team members and roles

- **Đoàn Hải Đăng** – Data pipeline (notebook Steps 0–3: loading the ATT&CK data, building the dataset, preprocessing, group-aware split)
- **Chu Văn Tuấn Đạt** – Modeling (Steps 4–7: baseline models, DistilBERT, threshold tuning, model comparison on validation)
- **Lê Đức Cường** – Final evaluation and risk analysis (Steps 8–10: test evaluation, error analysis, security impact analysis)
- **Pruvot Chales** – Interpretability, robustness and summary (Steps 11–13 and the final summary)
- **Nguyễn Phúc Thịnh** – Analysis supervision, overall summary and demo

Abstract

Security analysts read threat reports written in free text and must map each described behavior to MITRE ATT&CK. This project builds a multilabel classifier that reads one sentence describing attacker behavior and predicts which of the 15 ATT&CK Enterprise tactics (v20.0) it expresses. Labels come from the procedure examples published by MITRE, and the data are split by source group so that near-identical sentences never appear in both training and test data. A TF-IDF + Linear SVM baseline reaches a test Macro-F1 of 0.763, and a fine-tuned DistilBERT reaches 0.791 Macro-F1, 0.835 Micro-F1 and 0.781 exact match. The gain of +0.028 Macro-F1 has a 95% bootstrap interval of [+0.010, +0.046]. We analyze the errors, translate them into analyst workload and detection risk, test stability under input noise, and describe how the system could be deployed with human review.

1. Problem Definition

Threat and use case. Attackers' behavior is documented in CTI reports, vendor blogs and incident write-ups. To use that knowledge, a team maps each described behavior to ATT&CK, then compares the mapping with the detections it already has. This mapping is done by hand, report by report, and different analysts often label the same sentence differently. A delay or a missed mapping leaves a behavior without detection logic.

Actor. The user is a CTI analyst or detection engineer in a SOC or threat-intelligence team. The adversaries are the groups and malware families described in the reports. The system is decision support: it proposes tactics and the analyst accepts or corrects them.

Input. One sentence (or a short passage) of threat-report text, for example: "<ACTOR> used ntdsutil to obtain a copy of the victim environment ntds.dit file."

Desired output. The set of ATT&CK tactics the sentence expresses, with a probability for each, and the words that drove the prediction. The example above should return Credential Access.

ML formulation. Multilabel text classification with 15 labels. A tactic is the attacker's goal at a stage of an attack (for example Persistence), and a technique is the specific method used to reach it. One technique can serve several goals, so one sentence can carry several tactics: for example Scheduled Task belongs to Execution, Persistence and Privilege Escalation at once. In our data, 13.5% of the sentences carry more than one tactic.

Scope. Tactic-level classification of single sentences from ATT&CK Enterprise. We do not classify whole reports, ATT&CK Mobile or ICS, or the 600+ individual techniques.

2. Related Background and Choice of Approach

ATT&CK. MITRE ATT&CK is a public knowledge base of adversary behavior, organized in tactics (goals) and techniques (methods). Version 20 of the Enterprise matrix has 15 tactics. Compared with older releases, Defense Evasion has been replaced by two tactics, Stealth and Defense Impairment, so older papers and datasets use a different label space.

Text classification. Sparse bag-of-words features such as TF-IDF with a linear classifier are a standard, fast and interpretable baseline for text. Pretrained transformers such as BERT represent each word in context and are usually stronger when the same idea is expressed in different wording. DistilBERT is a smaller, faster distilled version of BERT, which makes it practical to fine-tune on a free GPU and to run on a CPU.

Multilabel learning. Each tactic is predicted by its own sigmoid output (or its own binary classifier), so several tactics can be active at once. Standard multilabel metrics are Micro-F1 (all label decisions pooled), Macro-F1 (average over labels, each label equal) and exact match (the whole label set is right).

Why this approach. The sentences are short and behavior-focused, the labels are expert-made, and rare tactics matter to defenders. A linear baseline gives a strong, transparent reference. A fine-tuned transformer tests whether context helps, especially for rare tactics. Related automated mapping efforts exist, for example MITRE's TRAM project, but we did not use them; the models here are trained and evaluated by the team on the data described below.

3. Dataset and Data Understanding

Source

MITRE ATT&CK Enterprise v20.0 in STIX 2.1 format (enterprise-attack-20.0.json, 26,690 objects), from the public repository mitre-attack/attack-stix-data. After removing revoked and deprecated entries, the file holds 706 techniques and sub-techniques. The data are public and may be used for academic work with attribution to MITRE.

Schema and label construction

A procedure example is the description of a “uses” relationship between a source (a group, malware, tool or campaign) and a technique. Each technique is mapped to its tactics through the kill_chain_phases field, and the sentence inherits all of those tactics as labels. Labels are therefore made by MITRE experts, not by keyword rules.

Column	Meaning
text_clean	Cleaned sentence (actor and indicators masked)
tactics	Labels: one or more of the 15 tactics
technique_ids	Technique(s) the sentence came from (not used as a feature)
sources	Group, malware, tool or campaign IDs that contain the sentence
group_id	Merged source group used for splitting
has_tech_name	True if the technique name appears in the sentence

Statistics

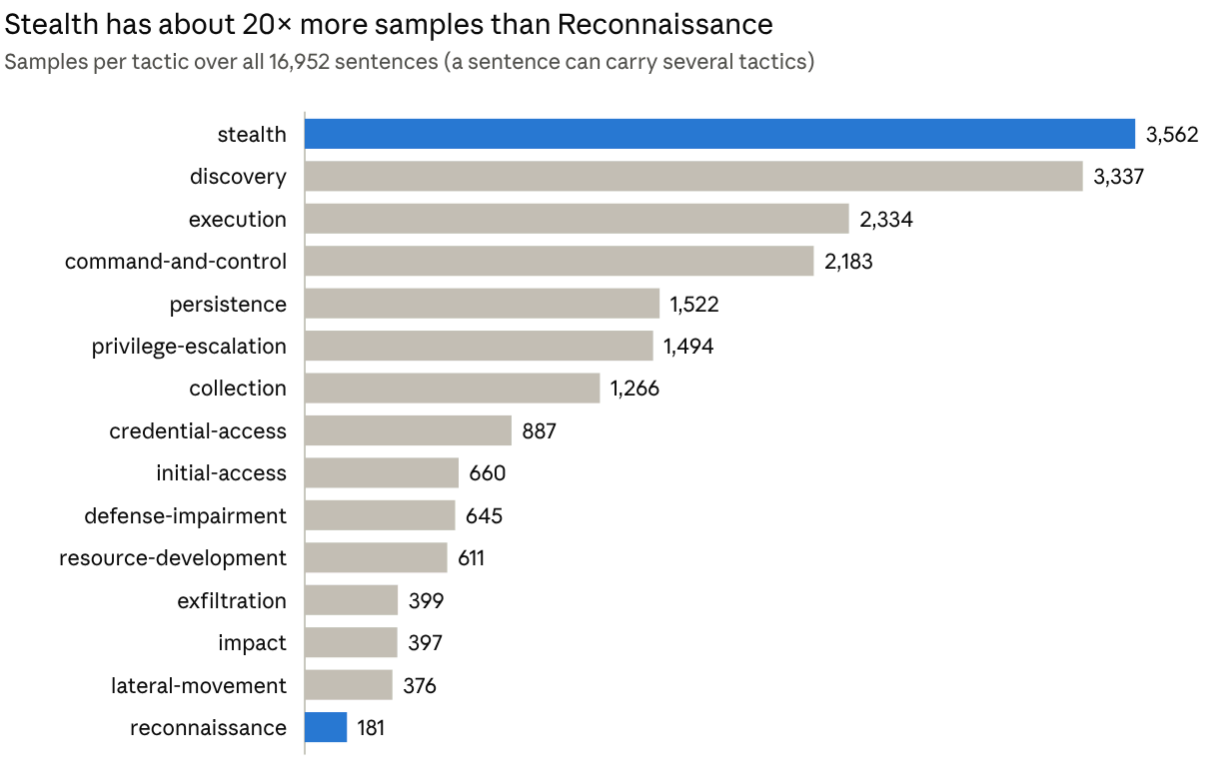
Item	Value
Raw procedure examples	17,636
After removing texts under 5 words and exact duplicates	17,563
Unique sentences after normalization and merging	16,952
Sources (malware, tools, groups, campaigns)	1,071

Item	Value
Average sentence length	about 14.6 words
Sentences with more than one tactic	13.5%
Sentences with more than one technique	0.6%
Sentences containing the technique name verbatim	13.8%
Sentences that came from more than one source	1.9%

Tactics per sentence: 14,662 sentences have 1 tactic, 1,839 have 2, 293 have 3, 156 have 4, and 2 have 5 or 6.

Class distribution

Tactic	Samples	Tactic	Samples
stealth	3,562	initial-access	660
discovery	3,337	defense-impairment	645
execution	2,334	resource-development	611
command-and-control	2,183	exfiltration	399
persistence	1,522	impact	397
privilege-escalation	1,494	lateral-movement	376
collection	1,266	reconnaissance	181
credential-access	887		



Computed from the final dataset of 16,952 sentences, 15 tactics

The distribution is strongly imbalanced: Stealth has about twenty times more sentences than Reconnaissance. A sentence can count for several tactics.

Data quality issues

- **Label noise.** Labels belong to the technique, not the sentence. A sentence that describes only one aspect still receives all tactics of its technique, and some sentences look like a different tactic (examples in Section 6).
- **Imbalance.** Rare tactics have few examples (Reconnaissance: 117 / 32 / 32 sentences in train / validation / test).
- **Duplicates and near-duplicates.** Sentences repeat across malware families, and sentences from one family are written alike. This drives the group-aware split.
- **Name leakage.** 13.8% of the sentences contain the technique name, which makes them easier. They are flagged and evaluated separately.
- **Multi-idea sentences.** Some passages combine several behaviors but carry one technique's labels.
- **Domain gap.** ATT&CK sentences are concise and edited. Real reports are longer and messier.
- **Label-space change.** Tactics changed between ATT&CK versions, so models do not transfer across versions without retraining.

4. Preprocessing and Feature Engineering

Cleaning

1. Remove citations and markup, such as (Citation: ...) and markdown links.
2. Remove technique IDs such as T1059 or T1059.001, which would reveal the label.
3. Mask the actor: the name of the group or malware that owns the sentence becomes the token <ACTOR>. The model cannot memorize that a given group always uses a given technique.
4. Normalize indicators of compromise: URLs, IP addresses, hashes, registry keys, file paths and e-mail addresses become <URL>, <IP>, <HASH>, <REGKEY>, <PATH> and <EMAIL>. Tool names and commands (powershell, schtasks, ntdsutil) are kept, because they are strong, legitimate signals.
5. Merge identical sentences from different sources into one row and unite their labels.
6. Flag, but do not delete, sentences that contain the technique name.

Encoding and feature extraction

- **Labels:** a 15-column multi-hot matrix, one column per tactic.
- **Baseline features:** TF-IDF with sublinear term frequency: word 1–2 grams (minimum document frequency 2) and, in the best configuration, character 3–5 grams (word-boundary, minimum document frequency 3, at most 150,000 features).
- **Transformer features:** the DistilBERT subword tokenizer, maximum length 128 tokens. No manual feature engineering is needed.

Handling imbalance

- Baseline: balanced class weights in each binary classifier.
- DistilBERT: weighted binary cross-entropy where the weight of the positive class is the square root of (negatives / positives) for each tactic, which is softer than full balancing.
- Splitting: tactic-aware, so every tactic appears in train, validation and test.
- Evaluation: Macro-F1 as the main metric, so rare tactics count as much as frequent ones.

Data splitting protocol

A random split of sentences would place very similar sentences from the same malware family in both training and test data and would inflate all scores. We split by group instead.

- Each sentence belongs to a group defined by its source. When a sentence has several sources, those sources are merged into one group with union-find. In total 1,071 sources became 690 groups.
- Groups are assigned to train, validation and test with a greedy algorithm that keeps the tactic distribution close to 70/15/15 and places groups holding the rarest tactics first. The seed is fixed (42).

Result: train 11,752 sentences (69.3%, 509 groups), validation 2,600 (15.3%, 91 groups) and test 2,600 (15.3%, 90 groups). For every tactic, its share in the test set lies between 14.8% and 17.7%. No group appears in more than one split, and only 0.4% of the validation and 0.6% of the test sentences have a TF-IDF cosine similarity of 0.9 or more to a training sentence.

Usage rules. Training data fit the models. Validation data are used for all choices: model, hyperparameters, thresholds and the best epoch. The test set was used once, at the end.

5. Modeling

Baseline

TF-IDF features with a one-vs-rest linear classifier, so each tactic has its own binary classifier. We compared Logistic Regression (C = 10) and Linear SVM (C = 0.5), both with balanced class weights, each with word n-grams only and with word plus character n-grams. If no tactic is predicted for a sentence, the highest-scoring tactic is assigned, because every sentence in the data has at least one. The best baseline on validation data is the **Linear SVM with word and character n-grams**.

Baseline (validation)	Micro-F1	Macro-F1	Exact match
Logistic Regression, word 1–2 grams	0.797	0.753	0.695
Linear SVM, word 1–2 grams	0.804	0.762	0.706
Logistic Regression, word + char	0.806	0.762	0.712
Linear SVM, word + char	**0.811**	**0.775**	**0.720**

Main model: fine-tuned DistilBERT

Setting	Value
Pretrained model	distilbert-base-uncased (Hugging Face)

Setting	Value
Output	15 logits with sigmoid, an independent probability for each tactic
Loss	Binary cross-entropy with positive weight sqrt(negatives / positives) per tactic
Optimizer	AdamW, learning rate 5e-5, weight decay 0.01, linear schedule with 10% warmup
Batch size / epochs	32 / 5
Maximum length	128 tokens
Hardware	Mixed precision on a Google Colab T4 GPU
Model selection	Epoch with the highest validation Macro-F1
Decision threshold	0.5 for every tactic (see Section 6)

Rationale. The sentences are short and describe behavior in varied wording, a setting where contextual representations can generalize beyond fixed keywords. DistilBERT keeps training and inference cheap. The weighted loss counters imbalance without the heavy over-prediction we saw with fully balanced weights in the baseline.

6. Evaluation

Protocol and metrics

We use a fixed group-aware train / validation / test protocol (Section 4). Cross-validation was used only for the honest threshold check on the validation set. The metrics are:

- **Macro-F1 (main).** Average F1 over the 15 tactics with equal weight. A rare tactic such as Reconnaissance or Impact matters to a defender as much as Discovery.
- **Micro-F1.** All label decisions pooled, dominated by large classes.
- **Exact match.** The whole label set of a sentence is right. A strict measure for multilabel sentences.
- **Label coverage.** Share of tactics predicted correctly at least once. It was 1.0 for all models, so it does not separate them.
- **Per-tactic precision, recall, F1 and confusion counts.** Precision relates to false alarms, recall to missed behaviors.

Accuracy is not used, because with 15 sparse labels a model can score very high accuracy while ignoring rare tactics.

Validation results

Model	Micro-F1	Macro-F1	Exact match
Best baseline (Linear SVM, word + char)	0.811	0.775	0.720
DistilBERT	**0.835**	**0.818**	**0.780**

DistilBERT improves Macro-F1 by 0.043 on validation and is better on 13 of 15 tactics, with the largest gains on Impact (+0.162), Reconnaissance (+0.103), Resource Development (+0.089) and Exfiltration (+0.087).

Threshold tuning

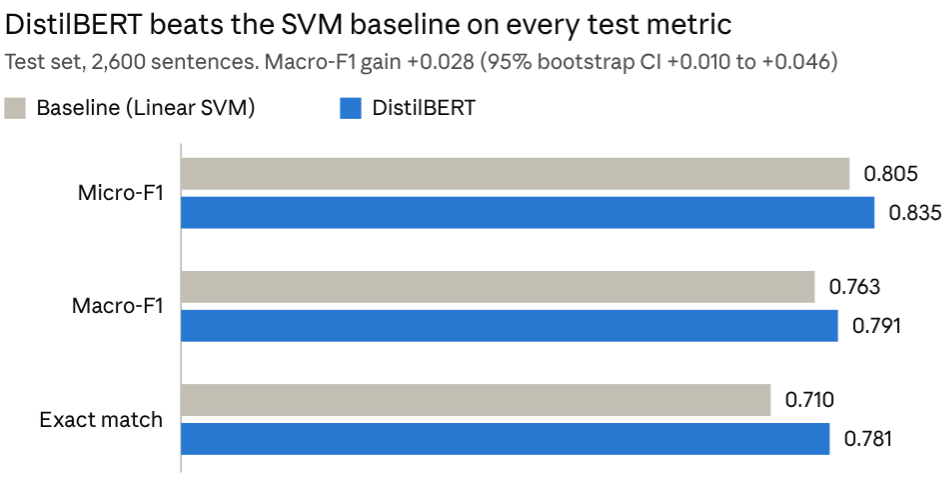
Strategy (DistilBERT, validation)	Micro-F1	Macro-F1	Exact match
Default threshold 0.5	0.835	0.818	0.780
Tuned on all validation data (optimistic)	0.843	0.824	0.807
Tuned with 2-fold grouped cross-validation (honest)	0.834	0.813	0.790

Tuning thresholds and scoring them on the same data looks like an improvement, but it is optimistic. When thresholds are tuned on one half of the validation set and scored on the other half (folds split by group), Macro-F1 falls below the default (0.813 versus 0.818), so thresholds are unstable, especially for rare tactics. **We kept 0.5.**

Final test results

The test set was evaluated once, with all choices frozen.

Model	Micro-F1	Macro-F1	Exact match	Label coverage
Baseline (Linear SVM, word + char)	0.805	0.763	0.710	1.0
DistilBERT	**0.835**	**0.791**	**0.781**	1.0



Final test results, evaluated once · 2,600 sentences

Bootstrap 95% intervals (1,000 resamples of the test sentences, seed 42) for Macro-F1 are [0.772, 0.807] for DistilBERT and [0.744, 0.780] for the baseline. The paired difference is +0.028 with an interval of [+0.010, +0.046], which excludes zero. DistilBERT is right and the baseline wrong on 341 sentences, the reverse on 158. Both models score slightly lower on test than on validation, as expected, because validation data selected the model and the epoch. The test figures are the ones we report.

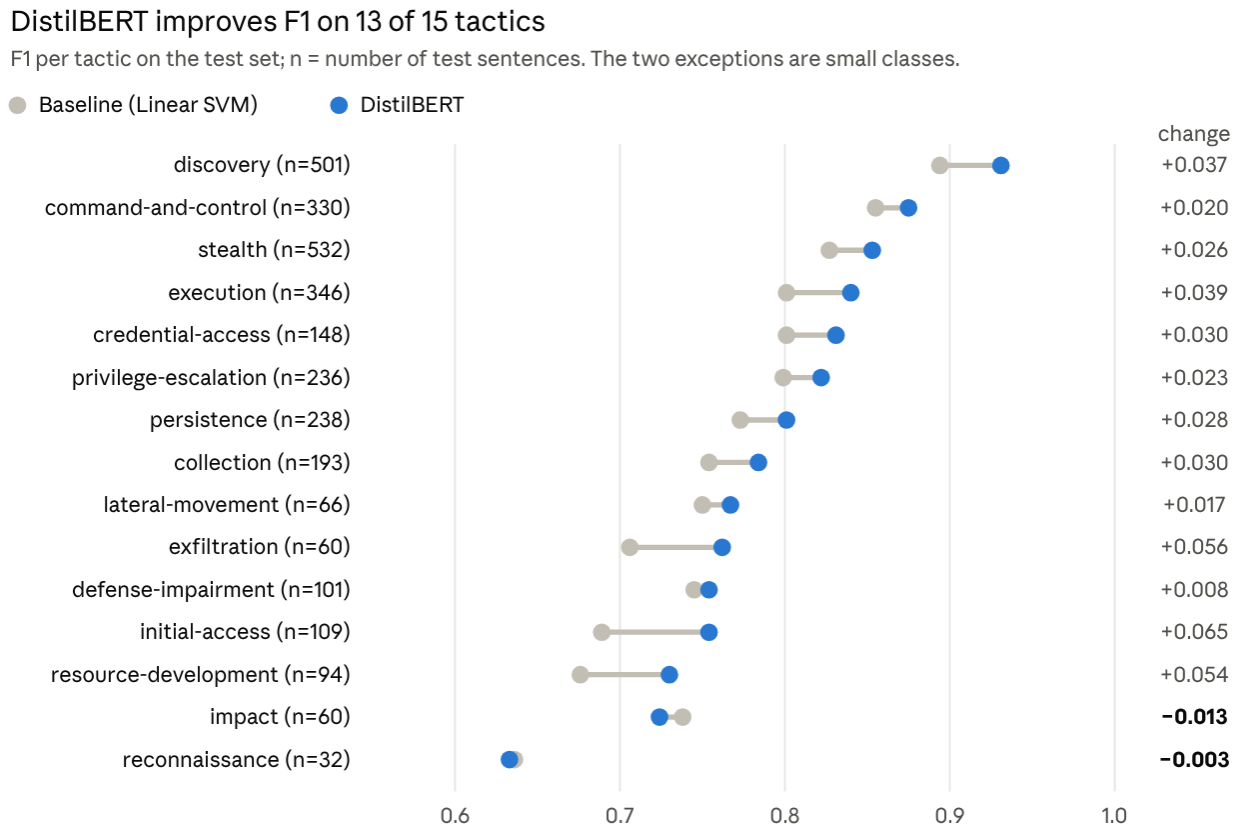
Leakage check. On the 2,218 test sentences that do not contain the technique name, DistilBERT reaches 0.826 Micro-F1, 0.770 Macro-F1 and 0.772 exact match (baseline: 0.797, 0.743 and 0.703). Performance stays close to the overall level, so the models do not rely on reading the technique name.

Confusion matrix and per-tactic results (test, DistilBERT)

For a multilabel problem the confusion matrix is computed for each tactic as a 2×2 table. TP is a correct tactic, FP a false alarm, FN a missed tactic and TN a correct rejection.

Tactic	TP	FP	FN	TN	Precision	Recall	F1	SVM F1
collection	149	38	44	2,369	0.797	0.772	0.784	0.754
command-and-control	293	47	37	2,223	0.862	0.888	0.875	0.855
credential-access	128	32	20	2,420	0.800	0.865	0.831	0.801
defense-impairment	78	28	23	2,471	0.736	0.772	0.754	0.745
discovery	480	50	21	2,049	0.906	0.958	0.931	0.894
execution	301	70	45	2,184	0.811	0.870	0.840	0.801
exfiltration	48	18	12	2,522	0.727	0.800	0.762	0.706
impact	46	21	14	2,519	0.687	0.767	0.724	0.738

Tactic	TP	FP	FN	TN	Precision	Recall	F1	SVM F1
initial-access	86	33	23	2,458	0.723	0.789	0.754	0.689
lateral-movement	51	16	15	2,518	0.761	0.773	0.767	0.750
persistence	191	48	47	2,314	0.799	0.803	0.801	0.773
privilege-escalation	201	52	35	2,312	0.794	0.852	0.822	0.799
reconnaissance	25	22	7	2,546	0.532	0.781	0.633	0.636
resource-development	73	33	21	2,473	0.689	0.777	0.730	0.676
stealth	462	89	70	1,979	0.838	0.868	0.853	0.827



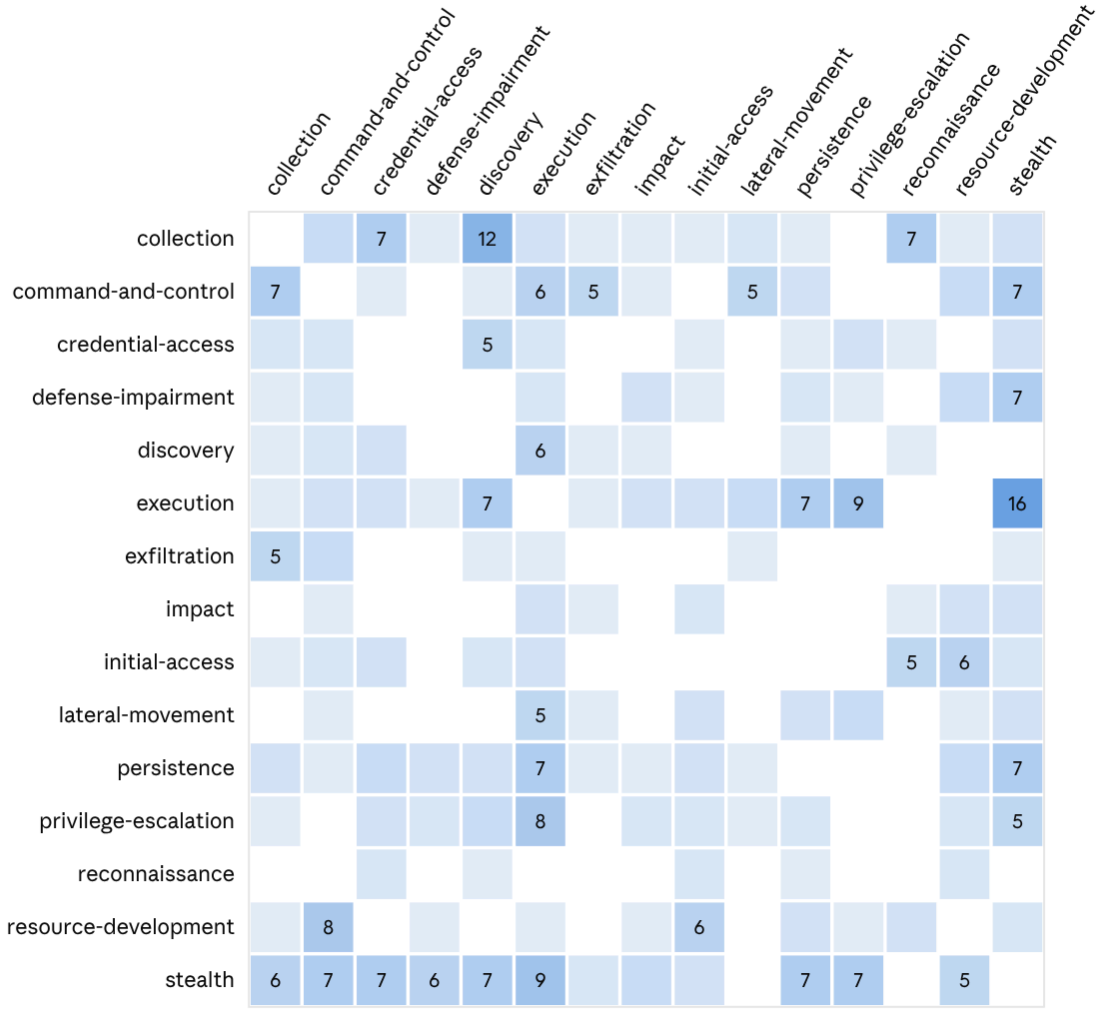
Per-tactic F1 on the test set · 2,600 sentences

DistilBERT is better on 13 of 15 tactics. The two exceptions, Impact and Reconnaissance, differ by very little and have only 60 and 32 test sentences, so their scores are noisy. Reconnaissance is the weakest tactic: recall is reasonable (0.781) but precision is low (0.532), because the model assigns this label too often.

Where the errors go: a tactic-to-tactic matrix shows which wrong tactic was predicted when a true tactic was missed.

Execution mistaken for Stealth is the most common confusion

Rows: true tactic that was missed. Columns: tactic predicted instead. Test set, DistilBERT, 448 confusions.



Test set, DistilBERT · counts of missed tactic and wrong tactic predicted instead

Error analysis

- **Several tactics are harder.** On the test set, exact match is 0.798 for sentences with one tactic, 0.655 for two and 0.628 for three.
- **Longer sentences are harder.** Exact match is 0.861 for up to 10 words, 0.775 for 11–20 words and 0.641 for 21–40 words (only 40 test sentences are longer than 40 words, too few to judge).
- **Confusions stay inside one family of tactics.** The most frequent test confusions are Execution predicted as Stealth (16 cases), Collection as Discovery (12), Execution as Privilege Escalation (9), Stealth as Execution (9) and Privilege Escalation as Execution (8). Many techniques legitimately belong to several of these tactics, and the boundary between hiding (Stealth) and disabling protection (Defense Impairment) is blurry.

- **Many confident errors are label noise.** On the validation set: “<ACTOR> has abused peer connections to exfiltrate data.” is labeled Defense Impairment although it describes exfiltration, and the model predicted Command and Control and Exfiltration. “<ACTOR> has attempted to collect browser information.” is labeled Collection, and the model predicted Discovery. “<ACTOR> has executed the Windows-native vssadmin command to create volume shadow copies.” is labeled Stealth, and the model predicted Execution. The maximum achievable score on this data is below 100%.

Interpretability

We explain predictions by occlusion: remove one word at a time and take the drop in the predicted probability of the tactic as that word’s importance. A small web demo shows the predictions and shades the important words. Results for five example sentences:

Sentence	Predicted	Most influential words
<ACTOR> used ntdsutil to obtain a copy of the victim environment ntds.dit file.	credential-access (0.94)	obtain (+0.13), copy (+0.10), ntds.dit (+0.05)
<ACTOR> has used scheduled tasks to establish persistence for installed tools.	execution (0.97), persistence (0.98), privilege-escalation (0.98)	none above 0.01
<ACTOR> enables SMBv1 during execution.	lateral-movement (0.96)	SMBv1 (+0.95)
<ACTOR> has attempted to collect browser information.	discovery (1.00)	browser (+0.01)
<ACTOR> can embed a copy of PsExec within its payload and place it in a temporary directory.	execution (0.89), privilege-escalation (0.59)	PsExec (+0.76), and (+0.42)

The model picks sensible technical terms (SMBv1, PsExec, ntds.dit). When a sentence contains several cues, no single word matters much (rows 1 and 2), because removing one word leaves enough evidence. The SMBv1 row shows near-total dependence on one keyword. The word “and” in the last row has a high score although it carries no meaning, which shows that occlusion measures influence on the prediction, not understanding. The browser row shows a confident wrong answer (the official label is Collection). Occlusion looks at one word at a time, so it misses word interactions.

7. Security Interpretation

The classifier supports an analyst who maps reports to ATT&CK. The two error types have different costs.

False negatives (missed tactics). A behavior stays unmapped, so a detection gap can go unnoticed. On the test set the model missed 434 of 3,046 true tactic labels (14.2%), and 14.8% of the sentences have at least one missed tactic. This is the more dangerous error, and recall per tactic is the number to watch.

False positives (extra tactics). The analyst checks the suggestion and rejects it, which costs time and, if frequent, erodes trust in the tool, so people start ignoring the output (alert fatigue). The model made 597 extra labels, about 23 per 100 sentences, and 20.3% of the sentences contain at least one wrong extra tactic. Because the tool is used offline on documents, a false alarm does not trigger an incident or block a user, so its cost is moderate compared with a SOC alert pipeline.

Recall level (test, DistilBERT)	Tactics
0.85 or higher, acceptable	discovery (0.958), command-and-control (0.888), execution (0.870), stealth (0.868), credential-access (0.865), privilege-escalation (0.852)
0.75 to 0.85, needs review	exfiltration (0.800), persistence (0.803), initial-access (0.789), reconnaissance (0.781), resource-development (0.777), lateral-movement (0.773), collection (0.772), defense-impairment (0.772), impact (0.767)
Below 0.75, risk	none

The tactics that matter most for late-stage damage are also the ones with few examples: Lateral Movement (15 missed), Impact (14 missed) and Exfiltration (12 missed). Their counts are small, so these figures are uncertain. The heaviest false-alarm burden is on large tactics (Stealth 89, Execution 70, Privilege Escalation 52), while Reconnaissance has the worst precision (22 false alarms against 25 correct).

Using confidence to focus review. On the validation set, 22.0% of the sentences have at least one wrong label. If the 20% of sentences with the lowest confidence (the probability of the weakest predicted tactic) are sent to a person, 66.7% of them are wrong and they contain 60.6% of all errors. With 30%, they contain 72.6% of all errors (53.3% of them wrong). Reading one fifth to one third of the sentences therefore catches most of the mistakes. These numbers come from validation data, and the rule should be re-checked on new data before use.

Operational conclusion. The tool is suitable for suggesting tactics and prioritizing review, not for automatic decisions. Missing tactics are best handled by showing the analyst the top suggestions with probabilities and by reviewing low-confidence sentences.

8. Robustness and Limitation Analysis

Robustness to input noise

We applied three mild perturbations to 500 random test sentences and re-ran the frozen DistilBERT model (a diagnostic only; nothing was retrained). The perturbations replace words with synonyms from a small dictionary, insert a typo in about 15% of words longer than four characters, and append the benign sentence "The report was published in 2020."

Input	Label sets unchanged	Micro-F1	Macro-F1
Original	100%	0.875	0.818
Synonym replacement	92.8%	0.855	0.798
Random typos	88.0%	0.831	0.767
Benign sentence appended	91.4%	0.863	0.799

Predictions stay unchanged for 88–93% of sentences. Synonyms and an extra sentence lower Macro-F1 by about 0.02, typos by 0.05, probably because subword tokenization splits misspelled technical terms into meaningless pieces. Compare the rows within this table only: this is a random subset of 500 sentences, so the original Macro-F1 (0.818) differs from the full test result (0.791). The synonym dictionary is small, so that row is optimistic. These are mild random changes, not a targeted attack.

Adversarial issues, poisoning and evasion

- **Evasion.** An attacker who controls the wording of a report, blog post or log message could rephrase it to avoid the right tactic. Our test did not try targeted rewording, and the SMBv1 example (one word carrying almost all of the score) suggests such an attack could work.
- **Poisoning.** The training data are public ATT&CK text, but if the system later learns from analyst corrections or from scraped reports, a malicious source could insert misleading examples. Feedback must be reviewed before it is used for retraining.

Drift

- **Vocabulary drift.** New malware, tools and cloud services appear constantly, and the model has not seen them.
- **Label drift.** ATT&CK itself changes: version 20 split Defense Evasion into Stealth and Defense Impairment, which a model trained on an older version could not predict. The model is tied to one ATT&CK version and must be retrained when the framework changes.
- **Source drift.** The training sentences are short and edited. Real reports are longer, narrative and noisy, so scores on real text are likely to be lower.

Label noise and generalization

- Labels belong to techniques, so many sentences have labels that do not match their wording (Section 6).
- Generalization to real reports (for example APTnotes) is untested. This is the biggest gap in the evidence.
- Rare tactics have only 32 to 60 test sentences, so per-tactic scores, especially for Reconnaissance, are unstable.
- Results come from one group-based split and one training run with seed 42, and we did not average over several runs.
- The classifier always returns at least one tactic, because every training sentence has one. Text that does not describe attacker behavior still receives a tactic. A “no tactic” class or a minimum confidence would be needed in practice.

Privacy

The project uses only public MITRE data. In practice, internal incident reports can contain confidential information such as hostnames, user names, customer data and indicators. The pipeline should run inside the organization, and masking of indicators (already part of our preprocessing) reduces exposure. Logs of inputs must follow the same retention rules as the source reports. Using an external API for inference would send that text to a third party and should be avoided for sensitive reports.

9. Deployment Thinking

Who uses it. CTI analysts and detection engineers, who review the suggestions; SOC managers, who use the aggregated mapping to see coverage gaps.

How the pipeline runs.

1. A report (PDF, HTML or text) is loaded and split into sentences.

2. Each sentence is cleaned the same way as the training data: IOCs are masked and technique IDs removed.
3. DistilBERT returns a probability for each of the 15 tactics. Tactics at or above the threshold are suggested, and the highest one is always shown.
4. The analyst sees the sentence, the suggested tactics with probabilities and the highlighted words. Sentences with low confidence are flagged for review.
5. The analyst accepts or corrects each suggestion, and the result goes to a coverage dashboard or the team's ATT&CK mapping.

Inference on single short sentences is light and can run on a CPU inside the organization. We did not measure latency, so this should be tested before commitment.

Logging. For every prediction store a timestamp, a document and sentence identifier (not the raw sensitive text, or the text under access control), the model and ATT&CK version, the probabilities, the suggested tactics, and the analyst's decision and correction.

Monitoring.

What to watch	Why	Warning sign
Analyst override rate per tactic	Direct measure of quality in production	Rate clearly above the rate measured on validation data (about 22% of sentences with an error)
Share of low-confidence sentences	Early drift signal	Share rising over weeks
Distribution of predicted tactics	Detects vocabulary or source drift	Large shift compared with the training distribution
Share of sentences with no tactic above the threshold	Detects text unlike the training data	Sudden increase
ATT&CK release notes	Label-space changes	New ATT&CK version published

Maintenance. Pin the ATT&CK version, store the model with its data and code version, and retrain when ATT&CK changes or when overrides show a sustained drop in quality. Corrections used for retraining should be reviewed first to avoid poisoning. Keep the TF-IDF baseline as a fallback and as a cheap sanity check.

Safeguards. Keep a human in the loop, show probabilities, add a "no tactic" option, and do not use the output for automatic blocking or any response action.

10. Conclusion and Future Work

What we did and learned. We built a reproducible pipeline that maps single CTI sentences to 15 ATT&CK tactics. With group-aware splitting and leakage controls, the TF-IDF + Linear SVM baseline reaches 0.763 Macro-F1 on test data and the fine-tuned DistilBERT reaches 0.791 Macro-F1, 0.835 Micro-F1 and 0.781 exact match. The gain is supported by a bootstrap interval, appears on 13 of 15 tactics and holds on sentences without the technique name. Per-tactic threshold tuning did not help when estimated honestly. We learned that evaluation design matters as much as the model: random splitting, name leakage and optimistic tuning would all have made the numbers look better than they are. Most remaining errors involve overlapping tactics, long multi-idea sentences and noisy official labels. Under mild random noise the predictions stayed stable for 88–93% of sentences, and typos were the most harmful change.

What remains incomplete.

- No evaluation on real threat reports (for example APTnotes) with manually labeled sentences.
- Tactic level only, not the 600+ techniques.
- One split, one seed, and one transformer; no security-specific language model was compared.
- No test of targeted adversarial rewording, and no “no tactic” option.
- Latency and production behavior were not measured.

Next steps.

- Build a small manually labeled test set from real reports and measure the domain gap.
- Extend to technique-level classification for frequent techniques, with a hierarchical prediction (tactic first, then technique).
- Compare with security-specific models and larger transformers, and average over several seeds.
- Add a rejection class or confidence threshold, evaluate the confidence-based review rule on new data, and compare occlusion with LIME or SHAP.
- Test targeted adversarial rewording and train with perturbed text for robustness.

Academic Integrity and Tool Declaration

- **Data:** MITRE ATT&CK Enterprise v20.0 (STIX 2.1), public data by MITRE, used with attribution.
- **Libraries:** Python, pandas, NumPy, SciPy, scikit-learn, PyTorch, Hugging Face Transformers, Matplotlib, Gradio.

- **Pretrained model:** distilbert-base-uncased from Hugging Face (Sanh et al., 2019), fine-tuned by the team on the data described above.
- **Generative AI:** a generative AI assistant (Claude, by Anthropic) was used while developing this project, for drafting code, analysis and parts of this report. [The team must state here exactly which parts it designed, implemented, ran, checked and wrote itself, and which parts were drafted with AI help, as the course requires.]
- **Team implementation work:** see the team members and roles listed at the top of this report.

Appendix A: Reproducibility

Code is organized as numbered scripts and one notebook. Running them in order reproduces the results (seed 42).

File	Purpose
01_build_dataset.py	Extract sentences, techniques and tactics from ATT&CK v20.0 STIX
02_preprocess.py	Clean text, mask actors and indicators, merge duplicates
03_split_data.py	Group-aware, multilabel-stratified 70/15/15 split with leakage checks
04_baseline.py	TF-IDF + Logistic Regression / Linear SVM baselines (validation only)
05_finetune_bert.py	Fine-tune DistilBERT (GPU recommended, for example Google Colab)
06_threshold_and_errors.py	Honest threshold tuning and validation error analysis
07_final_test.py	Single final evaluation on the test set with bootstrap intervals
08_explain_demo.py	Occlusion explanations, noise-robustness test and web demo
AIC211_Project_Topic10_ATTACK_Tactic_Pipeline.ipynb	The whole pipeline in one notebook

References

- MITRE ATT&CK Enterprise, version 20.0, STIX 2.1 data from the repository mitre-attack/attack-stix-data.
- Sanh, V., Debut, L., Chaumond, J., Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.
- Devlin, J., Chang, M., Lee, K., Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research.
- Course handout: AIC211 – Suggested Project Topics, Datasets, and Recommended Requirements, and the project template (Part A).